

LAPORAN PRAKTIKUM
JOB SHEET 06 - INHERITANCE (PEWARISAN)



Disusun Oleh:

Nama : Mohammad Hafidz Rafi' Rabbani
NIM : 254107020084
Kelas : 2H
No. Absen : 14
Repository : github.com/hafidzrafi/java-oop

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan	3
2. Praktikum	3
2.1. Percobaan 1: Dasar Inheritance dan Deklarasi Subclass (extends)	3
2.1.1. Pertanyaan	5
2.2. Percobaan 2: Pengaruh Access Modifier terhadap Keterwarisan Member	6
2.2.1. Pertanyaan	7
2.3. Percobaan 3: Penggunaan Kata Kunci super dan this	8
2.3.1. Pertanyaan	9
2.4. Percobaan 4: Rantai Konstruktor (Constructor Chaining) dan super()	10
2.4.1. Pertanyaan	11
2.5. Percobaan 5: Latihan Praktikum 1 (Hierarchical Inheritance: Karyawan, Manager, Staff)	13
2.5.1. Pertanyaan	14
2.6. Percobaan 6: Latihan Praktikum 2 (Multilevel Inheritance: StaffTetap dan StaffHarian)	15
2.6.1. Pertanyaan	16
3. Tugas	17
3.1. Tugas Mandiri: Sistem Tiket Transportasi Berbasis Hybrid Inheritance	17
3.2. Evaluasi Pertanyaan Tugas Mandiri	19

1. Tujuan

- Memahami konsep dasar pewarisan (Inheritance) dan mekanismenya pada bahasa pemrograman Java.
- Mampu membuat dan mendeklarasikan subclass dari suatu superclass menggunakan kata kunci `extends`.
- Memahami pengaruh access modifier (`public`, `protected`, default, dan `private`) terhadap keterwarisan atribut dan method.
- Menerapkan kata kunci `super` untuk mengakses atribut, method, dan memanggil konstruktor milik superclass.
- Memahami siklus hidup inisialisasi memori objek (object initialization lifecycle) dan alur eksekusi constructor chaining.
- Mengimplementasikan berbagai jenis inheritance: Single Inheritance, Multilevel Inheritance, Hierarchical Inheritance, dan Hybrid Inheritance.
- Membangun sistem studi kasus berbasis pewarisan dengan enkapsulasi data yang aman dan terstruktur.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: <https://github.com/hafidzrafi/java-oop>.

2. Praktikum

2.1. Percobaan 1: Dasar Inheritance dan Deklarasi Subclass (`extends`)

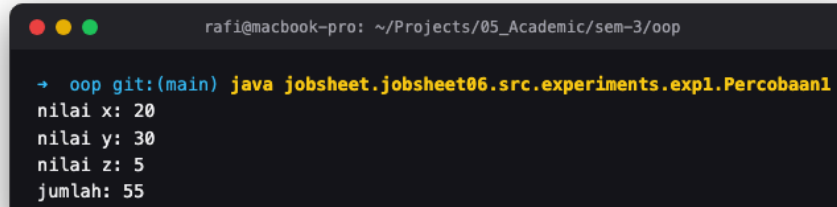
Pada percobaan pertama, dipelajari mekanisme dasar penurunan sifat (**Inheritance**) dari superclass `ClassA` ke subclass `ClassB` menggunakan kata kunci `extends`. Jika `ClassB` tidak mendeklarasikan `extends ClassA`, atribut `x`, `y`, dan method `getNilai()` milik `ClassA` tidak akan dikenal oleh `ClassB` dan memicu **compile error**.

```
// ClassB.java
package jobsheet.jobsheet06.src.experiments.expl;

public class ClassB extends ClassA {
    public int z;

    public void getNilaiZ() {
        System.out.println("nilai z: " + z);
    }
}
```

```
public void getJumlah() {  
    System.out.println("jumlah: " + (x + y + z));  
}  
}
```

A screenshot of a macOS terminal window. The title bar shows three colored window control buttons (red, yellow, green) and the text 'rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop'. The terminal content shows a command prompt with a blue arrow, followed by the command 'oop git:(main) java jobsheet.jobsheet06.src.experiments.exp1.Percobaan1'. Below the command, the output is displayed: 'nilai x: 20', 'nilai y: 30', 'nilai z: 5', and 'jumlah: 55'.

```
rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop  
  
→ oop git:(main) java jobsheet.jobsheet06.src.experiments.exp1.Percobaan1  
nilai x: 20  
nilai y: 30  
nilai z: 5  
jumlah: 55
```

2.1.1. Pertanyaan

1. Pada percobaan 1 di atas, program yang dijalankan mengalami error. Perbaikilah program tersebut agar dapat berjalan tanpa error!
 - Menambahkan kata kunci `extends ClassA` pada deklarasi `ClassB`:
`public class ClassB extends ClassA { ... }`. Dengan demikian, `ClassB` secara sah menjadi subclass dari `ClassA` dan mewarisi seluruh anggota class yang dapat diakses.
2. Jelaskan apa yang menyebabkan program pada percobaan 1 mengalami error!
 - Error terjadi karena `ClassB` pada awalnya belum dideklarasikan sebagai turunan dari `ClassA` (tanpa kata kunci `extends`). Akibatnya, `ClassB` tidak memiliki hubungan pewarisan dengan `ClassA`, sehingga variabel `x`, `y`, dan method `getNilai()` milik `ClassA` tidak dikenal di dalam `ClassB` maupun saat dipanggil melalui instance objek di `Percobaan1`.

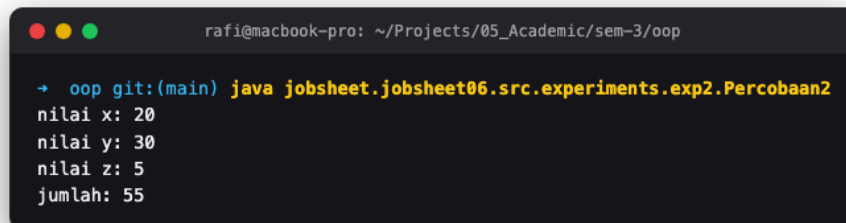
2.2. Percobaan 2: Pengaruh Access Modifier terhadap Keterwarisan Member

Percobaan kedua menguji batasan hak akses atribut pada hubungan pewarisan. Meskipun `ClassB` telah mewarisi `ClassA` (`ClassB extends ClassA`), atribut `x` dan `y` yang dideklarasikan dengan modifier `private` di dalam `ClassA` tidak dapat diakses secara langsung oleh `ClassB`. Masalah ini diselesaikan dengan mengubah modifier-nya menjadi `protected` atau menyediakan method getter/setter.

```
// ClassA.java
package jobsheet.jobsheet06.src.experiments.exp2;

public class ClassA {
    protected int x;
    protected int y;

    public void setX(int x) { this.x = x; }
    public void setY(int y) { this.y = y; }
    public void getNilai() {
        System.out.println("nilai x: " + x);
        System.out.println("nilai y: " + y);
    }
}
```



```
rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet06.src.experiments.exp2.Percobaan2
nilai x: 20
nilai y: 30
nilai z: 5
jumlah: 55
```

2.2.1. Pertanyaan

1. Pada percobaan 2 di atas, program mengalami error. Perbaikilah program tersebut agar dapat berjalan tanpa error!
 - Mengubah access modifier atribut `x` dan `y` pada `ClassA` dari `private` menjadi `protected`: `protected int x; protected int y;` (atau menyediakan method getter `getX()` dan `getY()` untuk diakses oleh `ClassB`).
2. Jelaskan apa yang menyebabkan program pada percobaan 2 mengalami error!
 - Error terjadi karena atribut `x` dan `y` di dalam `ClassA` dideklarasikan menggunakan access modifier `private`. Atribut `private` bersifat eksklusif hanya dapat diakses di dalam class tempat ia dideklarasikan dan **tidak pernah diwariskan ke subclass**. Ketika method `getJumlah()` di `ClassB` mencoba mengakses `x` dan `y` secara langsung (`x + y + z`), compiler menolak dengan pesan error **x has private access in ClassA**.

2.3. Percobaan 3: Penggunaan Kata Kunci super dan this

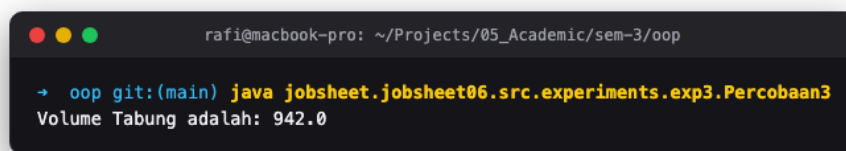
Percobaan ketiga mendemonstrasikan peran kata kunci `super` untuk merujuk pada member milik superclass (`Bangun`) dan kata kunci `this` untuk merujuk pada member milik class itu sendiri (`Tabung`). Class `Tabung` dapat langsung menghitung volume tabung dengan memanfaatkan atribut `phi` dan `r` yang diwariskan dari `Bangun`.

```
// Tabung.java
package jobsheet.jobsheet06.src.experiments.exp3;

public class Tabung extends Bangun {
    protected int t;

    public void setSuperPhi(double phi) { super.phi = phi; }
    public void setSuperR(int r) { super.r = r; }
    public void setT(int t) { this.t = t; }

    public void volume() {
        System.out.println("Volume Tabung adalah: " + (super.phi * super.r * super.r *
this.t));
    }
}
```



```
rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet06.src.experiments.exp3.Percobaan3
Volume Tabung adalah: 942.0
```

2.3.1. Pertanyaan

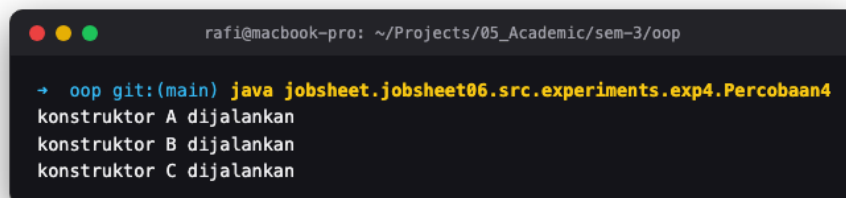
1. Jelaskan fungsi super pada potongan program method setter di class Tabung (setSuperPhi dan setSuperR)!
 - Keyword `super` digunakan untuk merujuk dan mengakses atribut `phi` dan `r` yang berada pada superclass (`Bangun`), sehingga nilai parameter yang diterima oleh setter di subclass `Tabung` disimpan langsung ke atribut milik class induk tersebut (`super.phi = phi;` dan `super.r = r;`).
2. Jelaskan fungsi super dan this pada potongan program method volume() di class Tabung!
 - `super` digunakan untuk merujuk secara eksplisit pada atribut `phi` dan `r` yang dideklarasikan pada superclass (`Bangun`), sedangkan `this` digunakan untuk merujuk pada atribut `t` (tinggi) yang dideklarasikan pada class saat ini (`Tabung`).
3. Jelaskan mengapa class Tabung tidak mendeklarasikan atribut phi dan r, tetapi class tersebut dapat mengaksesnya!
 - Karena class `Tabung` merupakan turunan langsung dari class `Bangun` (`public class Tabung extends Bangun`). Atribut `phi` dan `r` pada `Bangun` dideklarasikan dengan access modifier `protected`. Sesuai konsep **Inheritance**, semua atribut dan method bertipe `protected` otomatis diwariskan kepada subclass, sehingga `Tabung` dapat langsung menggunakannya tanpa deklarasi ulang.

2.4. Percobaan 4: Rantai Konstruktor (Constructor Chaining) dan super()

Percobaan keempat menganalisis urutan eksekusi konstruktor pada pewarisan bertingkat (**Multilevel Inheritance**): `ClassA` → `ClassB` → `ClassC`. Saat sebuah instance `ClassC` dibuat, Java mengeksekusi konstruktor superclass paling atas terlebih dahulu secara berantai sebelum mengeksekusi konstruktor milik subclass.

```
// ClassC.java
package jobsheet.jobsheet06.src.experiments.exp4;

public class ClassC extends ClassB {
    ClassC() {
        super(); // Pemanggilan eksplisit konstruktor parent (ClassB)
        System.out.println("konstruktor C dijalankan");
    }
}
```



```
rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet06.src.experiments.exp4.Percobaan4
konstruktor A dijalankan
konstruktor B dijalankan
konstruktor C dijalankan
```

2.4.1. Pertanyaan

1. Pada percobaan 4, tentukan class mana yang termasuk superclass dan subclass, lalu jelaskan alasannya!
 - `ClassA` adalah superclass bagi `ClassB`. `ClassB` adalah subclass dari `ClassA` sekaligus superclass bagi `ClassC`. `ClassC` adalah subclass dari `ClassB`. Hubungan ini membentuk hierarki pewarisan bertingkat (**Multilevel Inheritance**) karena `ClassC extends ClassB`, dan `ClassB extends ClassA`.
2. Ubahlah isi konstruktor default ClassC dengan menambahkan `super()` pada baris pertama, lalu amati apa yang terjadi!
 - Output program tidak mengalami perubahan sama sekali (**konstruktor A dijalankan, konstruktor B dijalankan, konstruktor C dijalankan**). Hal ini karena compiler Java secara otomatis menyisipkan statement `super();` implisit pada baris pertama setiap constructor subclass jika programmer tidak menuliskannya secara manual. Sehingga menulis `super();` secara eksplisit ataupun tidak menghasilkan bytecode JVM yang identik.
3. Ketika posisi `super()` dipindahkan ke baris kedua pada konstruktor default ClassC, terjadi error. Jelaskan mengapa error tersebut terjadi dan bagaimana urutan eksekusi konstruktor saat objek dibuat!
 - Error terjadi (**constructor call must be the first statement in a constructor**) karena spesifikasi JVM mewajibkan pemanggilan `super()` berada pada baris pertama constructor.

Alasan Arsitektural: Objek di Java dibangun dari dalam ke luar (**inside-out**). State dan alokasi memori milik superclass di heap memory **wajib selesai diinisialisasi secara sempurna terlebih dahulu** sebelum subclass diizinkan mengeksekusi instruksi logikanya sendiri. Jika subclass diizinkan mengeksekusi baris kode sebelum parent selesai dibuat, subclass berpotensi mengakses state parent yang belum diinisialisasi (**uninitialized state**).

Urutan Eksekusi (Constructor Chaining): Saat `new ClassC()` dipanggil, `ClassC()` memanggil `ClassB()`, `ClassB()` memanggil `ClassA()`, dan `ClassA()` memanggil `Object()`. Eksekusi body constructor berjalan dari root parent ke child terluar: `ClassA` selesai → `ClassB` selesai → `ClassC` selesai.

4. Apakah fungsi dari `super()` pada potongan kode di class `ClassC`?

- Fungsi `super()` pada `ClassC` adalah memanggil constructor milik superclass langsungnya (`ClassB`), memastikan proses inisialisasi state parent selesai dieksekusi sebelum body constructor `ClassC` dijalankan.

2.5. Percobaan 5: Latihan Praktikum 1 (Hierarchical Inheritance: Karyawan, Manager, Staff)

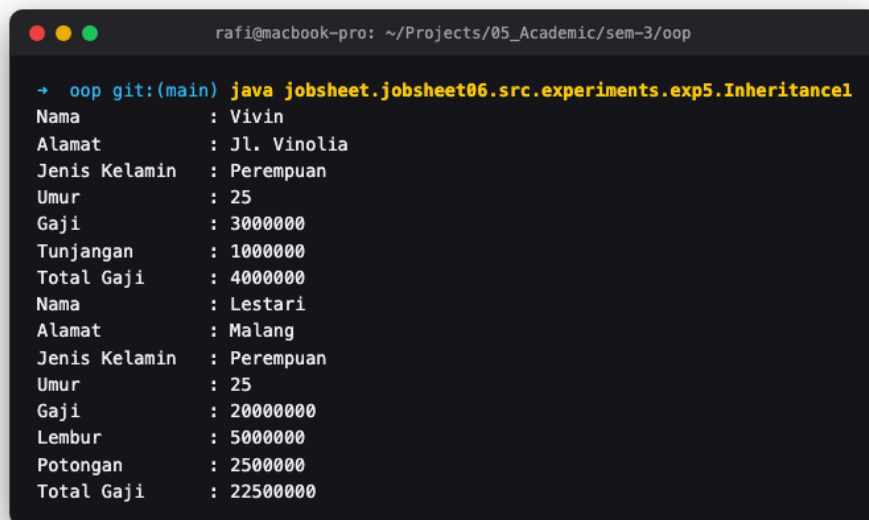
Percobaan kelima mengimplementasikan **Hierarchical Inheritance**, di mana satu superclass umum (`Karyawan`) diturunkan ke dua subclass sejajar (`Manager` dan `Staff`). `Manager` memiliki tunjangan tambahan, sedangkan `Staff` memiliki lembur dan potongan. Kedua subclass memanfaatkan `super.tampilDataKaryawan()` untuk menampilkan data dasar karyawan tanpa duplikasi kode.

```
// Manager.java
package jobsheet.jobsheet06.src.experiments.exp5;

public class Manager extends Karyawan {
    public int tunjangan;

    public Manager() {}

    public void tampilDataManager() {
        super.tampilDataKaryawan();
        System.out.println("Tunjangan      : " + tunjangan);
        System.out.println("Total Gaji      : " + (super.gaji + tunjangan));
    }
}
```



```
rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet06.src.experiments.exp5.Inheritance1
Nama      : Vivin
Alamat    : Jl. Vinolia
Jenis Kelamin : Perempuan
Umur      : 25
Gaji      : 3000000
Tunjangan : 1000000
Total Gaji : 4000000
Nama      : Lestari
Alamat    : Malang
Jenis Kelamin : Perempuan
Umur      : 25
Gaji      : 20000000
Lembur    : 5000000
Potongan  : 2500000
Total Gaji : 22500000
```

2.5.1. Pertanyaan

1. Identifikasi class mana yang merupakan superclass dan subclass pada Percobaan 5 di atas!
 - Superclass adalah `Karyawan`. Subclass adalah `Manager` dan `Staff`. (Class `Inheritance1` adalah driver utama pengeksekusi program).
2. Kata kunci apa yang digunakan untuk menurunkan suatu class dari class lain?
 - Kata kunci `extends`.
3. Perhatikan kode program pada class Manager. Atribut apa saja yang dimiliki oleh class tersebut? Identifikasi atribut mana saja yang diwarisi dari class Karyawan!
 - Atribut milik sendiri adalah `tunjangan`. Atribut yang diwarisi dari class `Karyawan` adalah `nama`, `alamat`, `jk`, `umur`, dan `gaji`.
4. Jelaskan penggunaan kata kunci super pada baris `super.tampilDataKaryawan()` dan `super.gaji + tunjangan` di class Manager!
 - `super.tampilDataKaryawan()` memanggil method display data pegawai dari superclass `Karyawan` agar `Manager` tidak perlu menulis ulang logika cetak identitas dasar. Sedangkan `super.gaji` mengakses nilai atribut gaji pokok dari superclass `Karyawan` untuk dijumlahkan dengan `tunjangan` dalam perhitungan total penghasilan.
5. Jenis inheritance apa yang ditunjukkan pada Percobaan 5 di atas? Jelaskan jawabanmu!
 - **Hierarchical Inheritance**, karena satu superclass (`Karyawan`) diturunkan secara sejajar ke lebih dari satu subclass yang berbeda peran (`Manager` dan `Staff`).

2.6. Percobaan 6: Latihan Praktikum 2 (Multilevel Inheritance: StaffTetap dan StaffHarian)

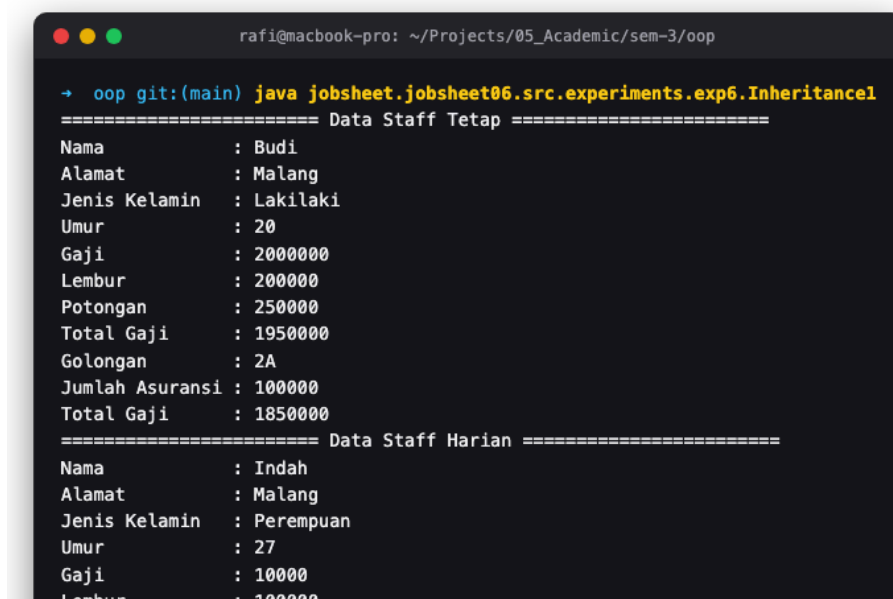
Percobaan keenam memperluas struktur pewarisan menjadi **Multilevel Inheritance**. Subclass `Staff` diturunkan kembali menjadi dua subclass yang lebih spesifik: `StaffTetap` (memiliki atribut golongan dan asuransi) serta `StaffHarian` (memiliki atribut jumlah jam kerja). Subclass generasi ketiga ini mewarisi seluruh atribut dari `Staff` sekaligus `Karyawan`.

```
// StaffTetap.java
package jobsheet.jobsheet06.src.experiments.exp6;

public class StaffTetap extends Staff {
    public String golongan;
    public int asuransi;

    public StaffTetap(String nama, String alamat, String jk, int umur, int gaji,
                      int lembur, int potongan, String golongan, int asuransi) {
        super(nama, alamat, jk, umur, gaji, lembur, potongan);
        this.golongan = golongan;
        this.asuransi = asuransi;
    }

    public void tampilStaffTetap() {
        super.tampilDataStaff();
        System.out.println("Golongan      : " + golongan);
        System.out.println("Jumlah Asuransi : " + asuransi);
        System.out.println("Total Gaji      : " + (gaji + lembur - potongan -
asuransi));
    }
}
```



```
rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet06.src.experiments.exp6.Inheritance1
===== Data Staff Tetap =====
Nama      : Budi
Alamat    : Malang
Jenis Kelamin : Laki-laki
Umur      : 20
Gaji      : 2000000
Lembur    : 200000
Potongan  : 250000
Total Gaji : 1950000
Golongan  : 2A
Jumlah Asuransi : 100000
Total Gaji : 1850000
===== Data Staff Harian =====
Nama      : Indah
Alamat    : Malang
Jenis Kelamin : Perempuan
Umur      : 27
Gaji      : 100000
Lembur    : 100000
```

2.6.1. Pertanyaan

1. Berdasarkan class-class di atas, mana yang merepresentasikan single inheritance dan mana yang merepresentasikan multilevel inheritance?
 - **Single Inheritance** direpresentasikan oleh relasi `Karyawan` → `Manager`. Sedangkan **Multilevel Inheritance** direpresentasikan oleh hierarki `Karyawan` → `Staff` → `StaffTetap` dan `StaffHarian`.
2. Perhatikan kode program class `StaffTetap` dan `StaffHarian`. Atribut apa saja yang dimiliki class-class tersebut? Identifikasi atribut mana yang diwarisi dari class `Staff` dan `Karyawan`!
 - Atribut milik sendiri: `StaffTetap` memiliki `golongan` dan `asuransi`, sedangkan `StaffHarian` memiliki `jamJamKerja`. Atribut yang diwarisi dari `Staff` adalah `lembur` dan `potongan`. Atribut yang diwarisi dari `Karyawan` adalah `nama`, `alamat`, `jk`, `umur`, dan `gaji`.
3. Apakah tujuan dari potongan kode `super(nama, alamat, jk, umur, gaji, lembur, potongan)` pada constructor `StaffHarian`?
 - Untuk meneruskan data parameter inisialisasi dari constructor `StaffHarian` ke constructor superclass `Staff`, yang kemudian akan diteruskan lagi ke constructor `Karyawan` guna menginisialisasi atribut warisan secara terpadu.
4. Apakah tujuan dari potongan kode `super.tampilDataStaff()` pada method `tampilStaffHarian()`?
 - Untuk memanggil method `display` pada superclass `Staff`, yang secara otomatis menampilkan rincian data karyawan dasar beserta data lembur dan potongan.
5. Pada class `StaffTetap`, atribut `gaji`, `lembur`, dan `potongan` dapat diakses langsung pada perhitungan gaji bersih. Mengapa hal ini dimungkinkan?
 - Hal ini dimungkinkan karena atribut `gaji`, `lembur`, dan `potongan` pada superclass (`Karyawan` dan `Staff`) dideklarasikan menggunakan access modifier `public` (atau `protected`). Berdasarkan prinsip **Multilevel Inheritance**, semua member yang tidak bersifat `private` akan terus diwariskan ke generasi subclass berikutnya (`StaffTetap`).

3. Tugas

3.1. Tugas Mandiri: Sistem Tiket Transportasi Berbasis Hybrid Inheritance

Tugas mandiri mengimplementasikan hierarki sistem tiket transportasi umum (`assignments`) yang mengombinasikan **Hierarchical Inheritance** dan **Multilevel Inheritance (Hybrid Inheritance)**:

- `Tiket`: Superclass umum (`kodeTiket`, `namaPenumpang`, `asal`, `tujuan`, `hargaDasar`).
- `TiketKereta`: Subclass dari `Tiket` (`nomorGerbong`, `nomorKursi`).
- `TiketPesawat`: Subclass dari `Tiket` (`maskapai`, `beratBagasi`, `hitungBiayaBagasi()`).
- `TiketDomestik`: Subclass dari `TiketPesawat` (`pajakBandara`).
- `TiketInternasional`: Subclass dari `TiketPesawat` (`nomorPaspor`, `asuransi`).
- `TestTiket`: Driver class pengujian seluruh skenario tiket.

```
// TestTiket.java
package jobsheet.jobsheet06.src.assignments;

public class TestTiket {
    public static void main(String[] args) {
        // Menggunakan no-argument constructor sesuai instruksi praktikum jobsheet
        TiketKereta tk = new TiketKereta();
        tk.kodeTiket = "KA-001";
        tk.namaPenumpang = "Andi";
        tk.asal = "Malang";
        tk.tujuan = "Jakarta";
        tk.setHargaDasar(350000);
        tk.nomorGerbong = 3;
        tk.nomorKursi = "12A";

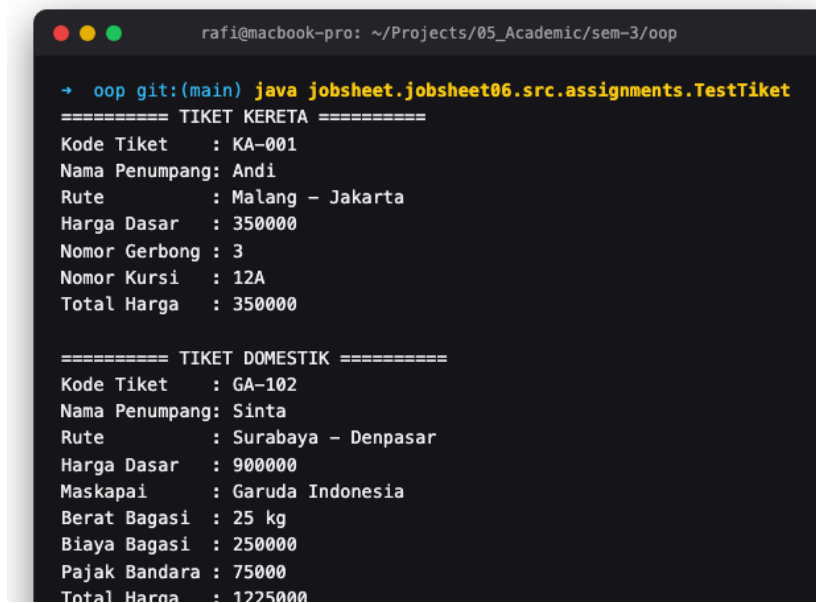
        TiketDomestik td = new TiketDomestik("GA-102", "Sinta", "Surabaya",
            "Denpasar",
            900000, "Garuda Indonesia", 25, 75000);
        TiketInternasional ti = new TiketInternasional("SQ-205", "Budi", "Jakarta",
            "Singapura",
            2500000, "Singapore Airlines", 20, "c1234567", 150000);

        System.out.println("\n===== TIKET KERETA =====");
        tk.tampilKereta();
        System.out.println("\n===== TIKET DOMESTIK =====");
        td.tampilDomestik();
        System.out.println("\n===== TIKET INTERNASIONAL =====");
        ti.tampilInternasional();
    }
}
```

```
}  
}
```

Aturan Bisnis Sistem Tiket:

1. **Biaya Bagasi** (`hitungBiayaBagasi()`): Bagasi pertama 20 kg gratis. Setiap kelebihan berat bagasi dikenakan biaya Rp 50.000 / kg.
2. **Formula Total Bayar:**
 - Kereta: Total = hargaDasar
 - Domestik: Total = hargaDasar + hitungBiayaBagasi() + pajakBandara
 - Internasional: Total = hargaDasar + hitungBiayaBagasi() + asuransi



```
rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop  
→ oop git:(main) java jobsheet.jobsheet06.src.assignments.TestTiket  
===== TIKET KERETA =====  
Kode Tiket : KA-001  
Nama Penumpang: Andi  
Rute : Malang - Jakarta  
Harga Dasar : 350000  
Nomor Gerbong : 3  
Nomor Kursi : 12A  
Total Harga : 350000  
  
===== TIKET DOMESTIK =====  
Kode Tiket : GA-102  
Nama Penumpang: Sinta  
Rute : Surabaya - Denpasar  
Harga Dasar : 900000  
Maskapai : Garuda Indonesia  
Berat Bagasi : 25 kg  
Biaya Bagasi : 250000  
Pajak Bandara : 75000  
Total Harga : 1225000
```

3.2. Evaluasi Pertanyaan Tugas Mandiri

Berikut adalah evaluasi atas empat pertanyaan analisis pada tugas mandiri:

a. Identifikasi Jenis Inheritance: Program mengimplementasikan **Hybrid Inheritance** (kombinasi dari **Hierarchical** dan **Multilevel Inheritance**):

- **Hierarchical Inheritance:** `Tiket` diturunkan ke `TiketKereta` dan `TiketPesawat`. Selain itu, `TiketPesawat` diturunkan ke `TiketDomestik` dan `TiketInternasional`.
- **Multilevel Inheritance:** Rantai bertingkat dari `Tiket` → `TiketPesawat` → `TiketDomestik` (serta `TiketInternasional`).

b. Daftar Atribut TiketInternasional Beserta Class Asalnya:

1. Dari superclass `Tiket`: `kodeTiket` (String), `namaPenumpang` (String), `asal` (String), `tujuan` (String), `hargaDasar` (int).
2. Dari superclass `TiketPesawat`: `maskapai` (String), `beratBagasi` (int).
3. Dari class `TiketInternasional` sendiri: `nomorPaspor` (String), `asuransi` (int).

c. Mengapa TiketDomestik Dapat Memanggil hitungBiayaBagasi():

Karena `TiketDomestik` adalah subclass turunan langsung dari `TiketPesawat` (`public class TiketDomestik extends TiketPesawat`). Method `hitungBiayaBagasi()` dideklarasikan dengan access modifier `public` di dalam class induk `TiketPesawat`, sehingga method tersebut secara otomatis diwariskan (**inherited**) dan dapat dipanggil langsung oleh class `TiketDomestik`.

d. Dampak Mengubah hargaDasar Menjadi private dan Solusinya:

- **Yang terjadi saat diubah ke private:** Program mengalami compile error (**hargaDasar has private access in Tiket**) karena atribut `private` tidak diwariskan dan tidak dapat diakses langsung oleh subclass (`TiketKereta`, `TiketDomestik`, `TiketInternasional`) saat melakukan kalkulasi total bayar.
- **Solusi tanpa mengubah kembali ke protected:** Menyediakan method getter `public int getHargaDasar() { return hargaDasar; }` dan setter `public void setHargaDasar(int hargaDasar)` pada class `Tiket`. Subclass kemudian mengakses nilai harga dasar melalui method `super.getHargaDasar()`. Cara ini justru lebih kokoh karena menerapkan prinsip enkapsulasi data murni.